

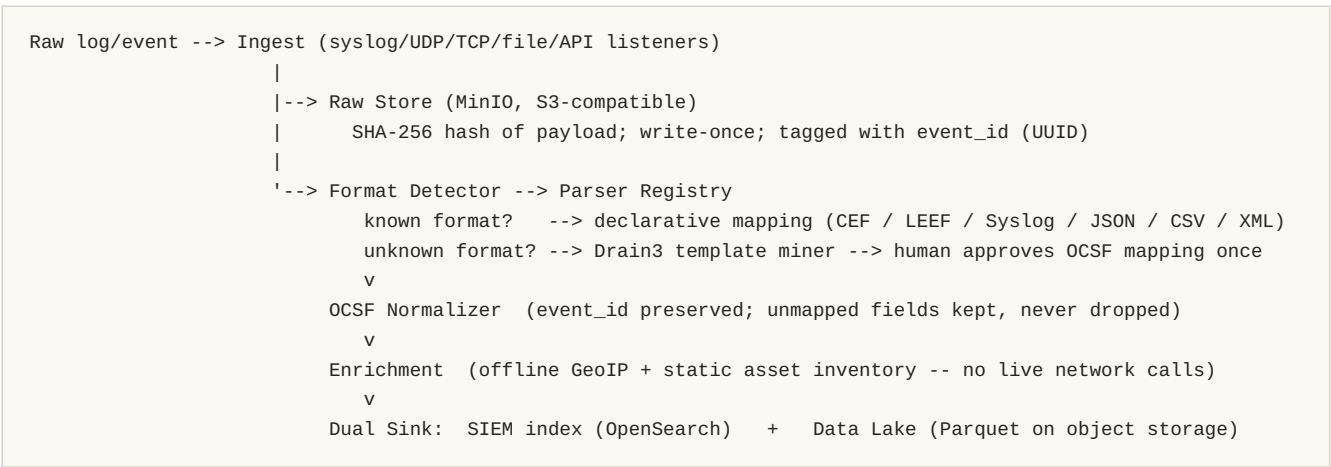
Technical Approach and Implementation

Universal Log Pre-processing Framework (ULPF) · National Technical Research Organisation (NTRO) · Software · Miscellaneous

1. Approach Overview

ULPF ingests perimeter network device logs and events in any format (Syslog, CEF, LEEF, JSON, XML, CSV, or proprietary), preserves the original event losslessly with a cryptographic hash, and normalizes it into the **OCSF** (Open Cybersecurity Schema Framework) taxonomy. Every normalized event carries a shared `event_id` back to its raw original, giving forensic-grade traceability. New log sources are onboarded through declarative, config-driven mappings — and when a genuinely unrecognized format appears, an online log-template miner (**Drain3**) auto-discovers its structure, needing only a one-time human field-mapping approval rather than hand-written parser code. The entire system runs in containers with zero runtime dependency on external networks, satisfying the air-gapped deployment requirement.

2. Architecture



2.1 Component Breakdown

COMPONENT	RESPONSIBILITY
Ingest	Accepts syslog/UDP/TCP/file/API input from any perimeter device.
Raw Store	Hash-tagged, write-once original event, joined by <code>event_id</code> .
Format Detector	Identifies CEF / LEEF / Syslog / JSON / CSV / XML on sight.
Parser Registry	Declarative per-vendor mapping files; no hardcoded per-vendor logic.
Drain3 Fallback	Auto-mines templates for unrecognized/proprietary formats.
OCSF Normalizer	Maps parsed fields onto real OCSF field names.
Enrichment	Offline GeoIP + static asset-inventory lookup, zero network calls.
Dual Sink	Writes to OpenSearch (SIEM) and Parquet (data lake) simultaneously.

2.2 Technology Stack

Python 3.11+

Drain3

MinIO

OpenSearch

Apache Parquet / pyarrow

FastAPI

Docker Compose

OCSF schema

EVENT ENVELOPE (NORMALIZED DOCUMENT SHAPE)

```
event_id, ingest_timestamp,
source: {vendor, product, detected_format, collector_host},
raw: {bucket, object_key, sha256, byte_length},
ocsf: { ... mapped fields ... },
parsing: {parser_used, template_id, confidence, unparsed_fields}
```

3. Requirement-to-Implementation Mapping

PS REQUIREMENT	HOW ULPF IMPLEMENTS IT
(a) Preserve raw data, no loss	

	Hash-tagged write-once raw store; unmapped fields kept in <code>parsing.unparsed_fields</code> , never dropped.
(b)/(c) Extract & normalize into common taxonomy	Format-specific parsers feeding a shared OCSF normalizer.
(d) Traceability raw ↔ normalized	Shared <code>event_id</code> joins every normalized document to its raw original.
(e) Plug-and-play onboarding	Declarative mapping files for known formats; Drain3 auto-mining + one-time human approval for new ones.
(f)/(g) Unified visibility & SIEM/data-lake integration	Dual sink into OpenSearch (SIEM-style search) and Parquet (data lake) simultaneously.
(h) AI/ML-ready analytics	OCSF's typed, enumerated fields are directly consumable by downstream ML pipelines.
(i) Reduced parser development effort	Drain3 template mining replaces hand-written parsers for unseen formats.
(j) Air-gapped deployment	All services (MinIO, OpenSearch, pipeline, UI) run offline with zero runtime external calls.
(k) Container packaging	Single docker-compose bundle; platform-independent.

4. Implementation Plan

PHASE	DELIVERABLE	ACCEPTANCE CHECK
0. Scaffold	Repo structure, config, CI-ready tests	<code>pytest</code> runs; <code>docker-compose config</code> validates.
1. Raw store	Hash-tagged ingest & storage	Round-trip test: bytes out equal bytes in, hash verified.
2. Parsers	Detector + mapping per known format	One passing test per real sample file, asserting correct fields.
3. Normalizer	OCSF mapping layer	Same logical field lands under identical OCSF name across 3+ vendors.
4. Drain3 fallback	Auto template mining + approval step	Held-out format converges to a stable template; next log auto-parses.
5. Enrichment	Offline GeoIP + asset lookup	Known internal IP resolves to asset name; public IP resolves to geo, offline.
6. Dual sink	OpenSearch + Parquet writers	Same <code>event_id</code> findable in both stores after a batch ingest.
7. API/UI	FastAPI + minimal demo UI	Raw vs normalized view for any <code>event_id</code> , hash visibly matching.
8. Containerization	docker-compose, air-gap proof	Full pipeline functions with host network disconnected.
9. Load test	Throughput measurement script	Real, reproducible events/sec figure recorded.
10. Deliverables	README, this document, slide deck, demo script	All claims verified against working code.

5. Validation & Datasets

Parsers are built and tested against real sample logs (Cisco ASA, Palo Alto, Fortinet, Checkpoint from `elastic/beats` filebeat modules; RFC 3164/5424 syslog; Suricata `eve.json`; Zeek connection logs), with one dataset from **Loghub** deliberately held out and used only to test Drain3's live auto-onboarding. **CIC-IDS2017** and **UNSW-NB15** supply labelled attack traffic for the downstream analytics/ML demonstration layer.